

COMPETITION EXTENDED ABSTRACT

LAYOUTLAB AR

A NOSS-Aligned Augmented Reality Trainer for Java Swing Interface Prototyping

Application Prototype Development • IT-010-3:2016-C01

Mohd Azlan bin Ab Aziz¹ | Hasnah binti Ngah²

¹ Jabatan Teknologi Maklumat & Komunikasi • ² Jabatan Perdagangan
Politeknik Mukah, Sarawak, Malaysia

Extended Abstract

Novice Java learners often understand Swing syntax before they understand the invisible rules that govern interface layout. This gap produces a costly trial-and-recompile cycle: components disappear, resize unexpectedly, or compete for the same BorderLayout region, while students change code without developing a transferable mental model. LayoutLab AR addresses this problem through an assessed, browser-based augmented reality module aligned to IT-010-3:2016-C01, Application Prototype Development. Using an iPhone camera and an original tracked target, learners view a virtual JFrame anchored in physical space and complete three sequential missions by touching the 3D model itself. They place an application title in NORTH, predict that CENTER absorbs remaining space during resizing, and diagnose why two buttons cannot independently occupy SOUTH before constructing the correct nested JPanel repair. The completed AR state is converted into deterministic, compilable Java evidence. The learning sequence integrates text, tracked 3D graphics, animation, bilingual spoken instructions, direct spatial interaction and generated source code—well beyond the organiser’s minimum requirement of three media elements. The wider LayoutLab platform also supports BorderLayout, FlowLayout and GridLayout exploration, but the competition module intentionally concentrates on one assessable BorderLayout competency. Two earlier formative evaluations of the browser platform (n=11 and n=6) established the learning need and shaped the interface; they are not presented as proof of AR learning effectiveness. The AR build has instead completed technical acceptance on an iPhone 13 over production HTTPS, including stable target tracking, all three missions, audio and final Java evidence. Seventy-five automated tests currently protect the engine, mission logic and code-generation path. LayoutLab AR therefore presents a focused TVET innovation: learners do not merely watch an AR visualisation—they construct, test and repair an application prototype through it.

Keywords: augmented reality, BorderLayout, deterministic assessment, Java Swing, NOSS, prototype development, TVET

NOSS Code	IT-010-3:2016 — Pembangunan Aplikasi, Tahap 3
Competency Unit	IT-010-3:2016-C01 — Application Prototype Development
Primary Work Activity	Implement Application Prototype Mock-Up Flow
Supporting Activity	Conduct User Interface and User Experience Functionality Test
Assessed Scope	Construct, test and diagnose a Java Swing BorderLayout prototype in AR

1. Background and Problem Statement

In Swing, a learner does not freely place each component. A layout manager interprets constraints, preferred sizes, available space and nesting. BorderLayout is especially revealing: each of its five regions accepts one direct component; NORTH and SOUTH take height, EAST and WEST take width, and CENTER absorbs what remains. When two components are added directly to the same region, only the last is displayed. Conventional slides can state these rules, but they do not let every learner manipulate and test them repeatedly.

The first formative evaluation of the earlier browser platform involved 11 Semester 5 students. Most rated their Java ability at 2 or 3 out of 5, and 81.9% reported sometimes or often struggling to arrange components as intended. Their difficulties included choosing a layout manager, predicting resize behaviour and understanding nested panels. These findings support the need for a visible, hands-on learning sequence; they do not, on their own, prove the effectiveness of the later AR module.

2. Innovation Objective

The primary objective is to enable learners to implement and functionally test a Java Swing application prototype by manipulating the layout rules in augmented reality. The module is designed to:

- make an invisible layout algorithm observable as a stable, tracked 3D model;
- require learners to apply, test and diagnose BorderLayout rules through direct spatial interaction;
- connect each AR decision to deterministic Java source code that can be inspected and compiled; and
- produce auditable evidence for the specified NOSS competency and work activities.

Strategic scope decision

The competition module deliberately assesses BorderLayout only. FlowLayout and GridLayout remain available as enrichment in the wider LayoutLab Playground. This narrower scope gives the AR experience a complete learning arc—construct, predict, test, diagnose and repair—instead of presenting three shallow demonstrations.

3. Description of the Innovation

LayoutLab AR runs in a mobile browser over HTTPS. The learner points an iPhone camera at an original LayoutLab target card. Once detected, the application anchors a virtual JFrame and five colour-coded BorderLayout regions to the target. Touch coordinates are raycast against the tracked 3D meshes; therefore, the learner cannot complete the assessed flow through a conventional multiple-choice control. Tracking, spatial anchoring, 3D rendering and direct touch are essential to the task.

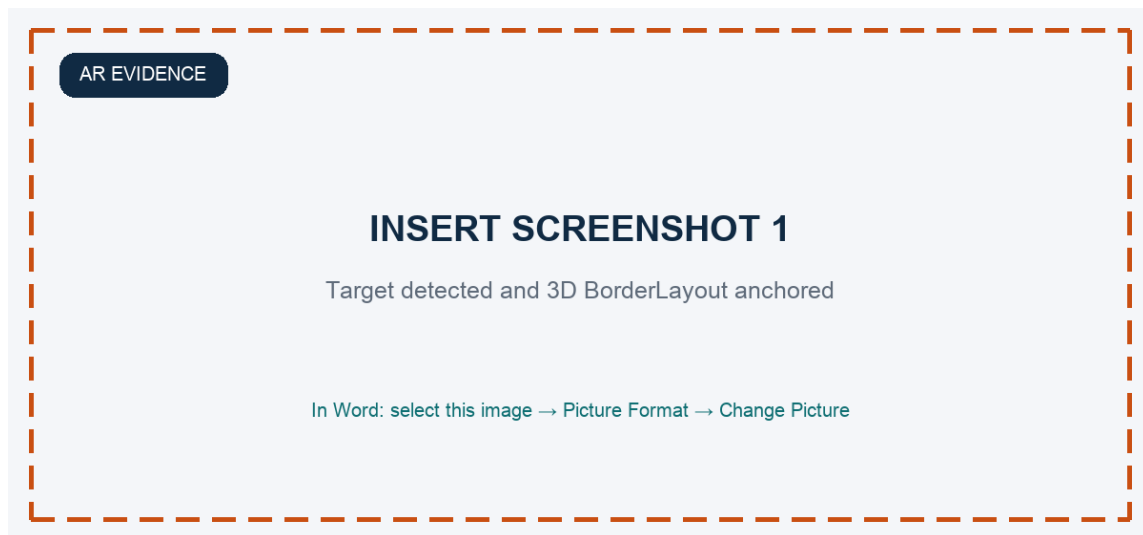


Figure 1. Replace with an iPhone screenshot showing the complete target, 'Target found' status and the anchored 3D BorderLayout model.

4. Three-Mission Assessed Learning Flow

Mission	Required AR action	Competency demonstrated	Evidence produced
1 — Construct	Touch NORTH on the tracked 3D frame to place the application title.	Select and apply the correct BorderLayout constraint.	Title appears in NORTH; state and Java are updated.
2 — Test	Touch the region expected to absorb space, then observe the frame resize.	Predict and verify dynamic layout behaviour.	CENTER stretches through a user-triggered animation.
3 — Diagnose and repair	Touch SOUTH to reveal the collision; activate the AR repair hotspot.	Identify the one-component-per-region rule and create a nested structure.	A JPanel containing Save and Cancel occupies SOUTH; score reaches 3/3.

Mission 1 — Construct the prototype

The learner is asked to place an application title that must remain across the top of the interface. Tapping the NORTH mesh changes the tracked model and advances the mission. A wrong region does not produce a generic failure message; the learner remains in the mission and can try again.

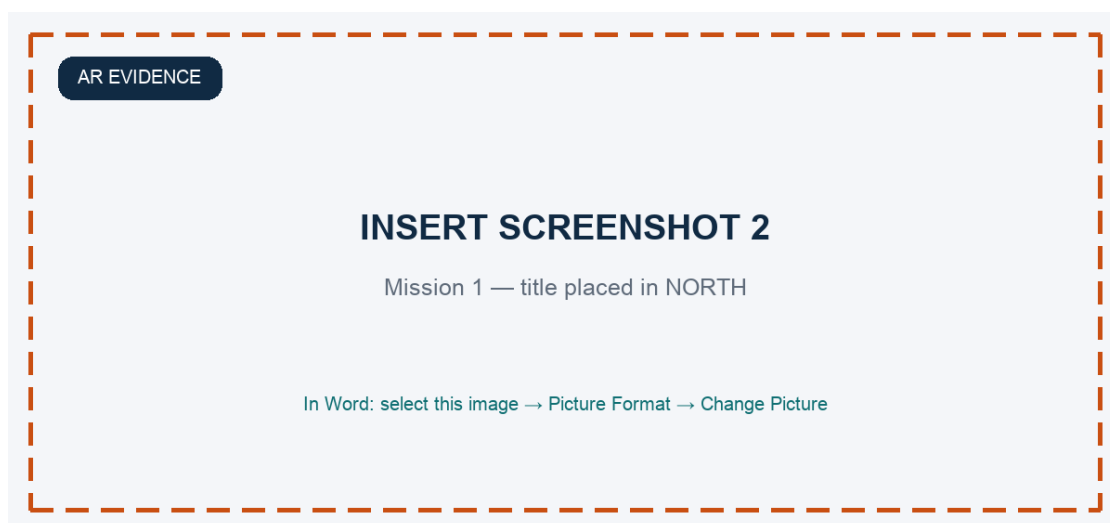


Figure 2. Replace with the successful Mission 1 state: application title placed in NORTH and progress shown as 1/3.

Mission 2 — Conduct a functionality test

The learner predicts which region changes when the JFrame becomes larger. Selecting CENTER triggers a visible resize animation and the rule ‘CENTER absorbs the remaining space’. This links a design decision to observable interface behaviour and directly supports UI functionality testing.

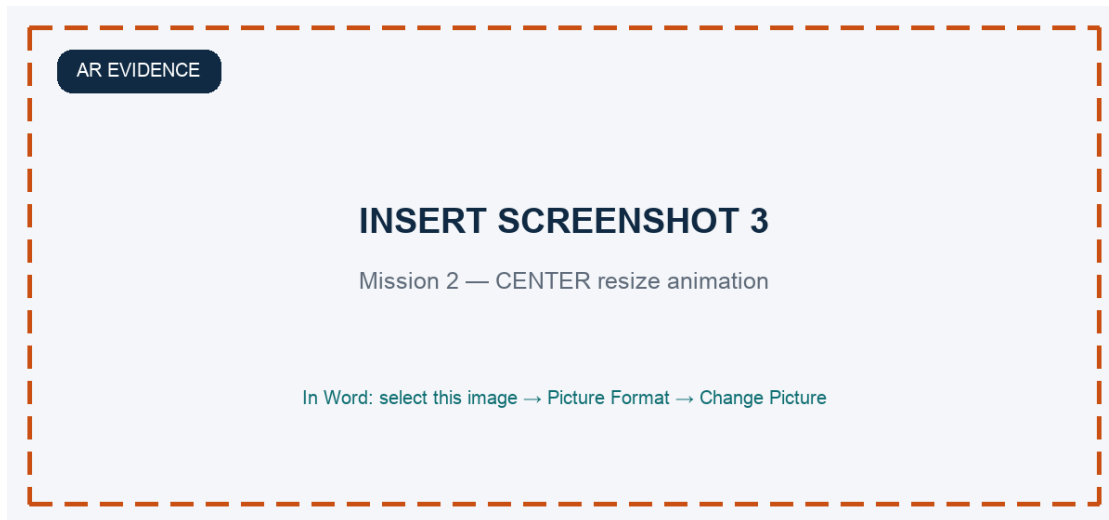


Figure 3. Replace with the resize sequence or final state showing CENTER stretched and progress shown as 2/3.

Mission 3 — Diagnose failure and implement the structural repair

The final mission reproduces a common Swing defect: Save and Cancel are both added directly to SOUTH, so only the last component remains visible. Touching SOUTH activates an X-ray view that reveals the collision. The learner must then activate a prominent repair hotspot on the tracked model. The system constructs a nested JPanel containing both buttons and places that single panel in SOUTH.

Why this is more than a visual effect

The learner must first identify the failed region, reveal the hidden structure and then choose a structural correction. The AR object is simultaneously the model, the interaction surface and the assessment interface.

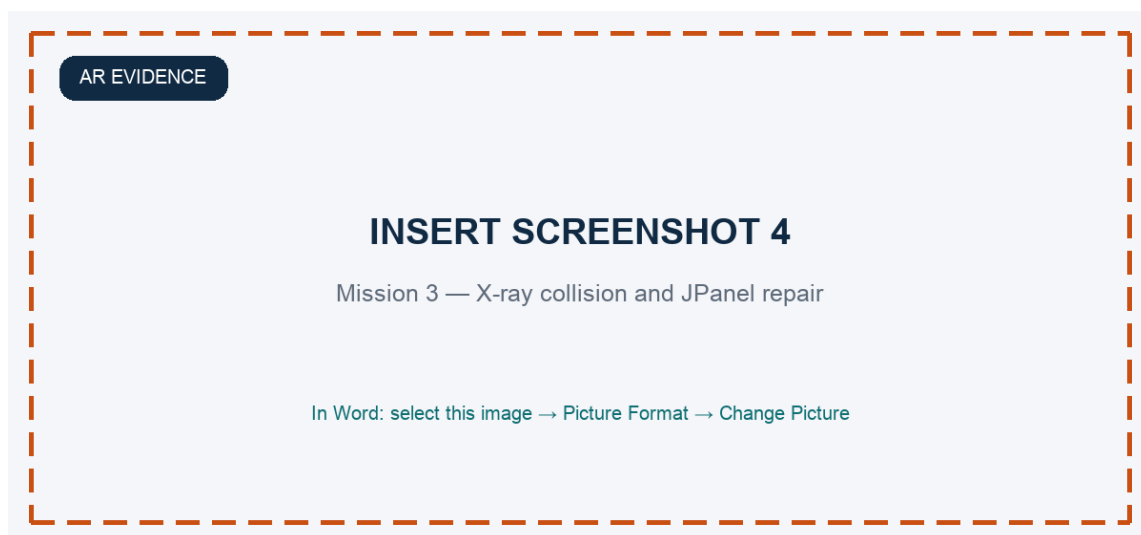


Figure 4. Replace with evidence of the SOUTH X-ray collision and/or the repaired ‘JPanel: Save + Cancel’ state.

Generated Java as competency evidence

After completion, the final AR state is converted through the same deterministic component-tree and Java-generation path used by LayoutLab. The evidence must show one JPanel added to BorderLayout.SOUTH, with Save and Cancel added inside that panel. Generative AI is not used to decide correctness or generate this assessed output.



Figure 5. Replace with the 3/3 completion state and the expanded Java evidence showing the nested JPanel repair.

5. AR Media Elements and Learning Function

The organiser indicated that AR should be the focus and that at least three media elements must be present. LayoutLab AR integrates six elements, each with a defined learning purpose rather than using media as decoration.

Element	Implementation in LayoutLab AR	Learning or evidence function
Text	Mission prompts, NOSS mapping, Swing rules, feedback and generated Java.	Explains the task and records technical evidence.
Tracked 3D graphics	Virtual JFrame, five regions and components anchored to the target.	Makes the abstract layout model spatially visible.
Animation	CENTER resize and SOUTH collision/X-ray transitions.	Shows change over time and exposes hidden behaviour.
Audio	User-triggered Bahasa Malaysia or English spoken instructions.	Supports multimodal guidance without replacing the visual task.
Direct interaction and assessment	Touch/raycast selection on mission-relevant 3D objects.	Requires the learner to act on the AR model to progress.
Generated code	Deterministic javax.swing/java.awt source from the final AR state.	Connects prototype behaviour to verifiable implementation evidence.

6. Methodology and Development Process

Development followed an iterative design–build–test cycle. The earlier browser platform established the instructional need and supplied the deterministic layout engine. The competition build then reframed one high-value misconception as a three-mission AR sequence.

Stage	Work completed	Decision or result
Formative iteration 1 (n=11)	Surveyed difficulty, conceptual clarity, confidence and feature requests.	Prioritised live visualisation, copyable Java, clearer guidance and structured practice.
Formative iteration 2 (n=6)	Evaluated the revised browser platform and challenge modes.	Confirmed ‘aha’ moments around automatic placement and resizing; identified onboarding and sizing refinements.
AR P0	Built tracked 3D BorderLayout model, mission state machine, text, animation and audio.	Established a complete NOSS-aligned construct–test–diagnose flow.
AR P1/P2	Replaced answer buttons with raycast interaction; strengthened mission 3 and evidence reveal.	Made AR essential to completion and improved reliability on an iPhone screen.
Device acceptance	Tested production HTTPS deployment on iPhone 13.	Stable tracking, three missions, completion state, audio and Java evidence verified.

Original Tracking Target

The target is an original LayoutLab-branded card with dense, asymmetric detail across the complete frame. It was compiled with the official MindAR 1.2.5 image-target compiler. Acceptance evidence records a resolution of 1448 × 1086 pixels, 12 scale keyframes and 4,656 detected feature points (3,496 maxima and 1,160 minima). These characteristics improve recognition and reduce ambiguity during the assessed interaction.

7. Findings, Validation and Evidence

The evidence is intentionally separated into formative learning evidence and technical acceptance evidence. This prevents the student surveys for the wider browser platform from being misrepresented as a controlled evaluation of the new AR module.

Evidence	Result	What it supports—and what it does not
Iteration 1 student survey	n=11; 81.8% agreed/strongly agreed that visualisations clarified layout-manager differences; 81.8% reported greater confidence building a GUI from scratch.	Supports the learning need and acceptance of visual manipulation. It is not a controlled pre/post result.
Iteration 2 student evaluation	n=6; students described immediate placement, manager rules and dynamic resizing as learning breakthroughs.	Supports iterative design decisions for the wider LayoutLab platform, not causal AR effectiveness.
Automated quality gate	75 automated tests currently pass across engine, mission logic, grading and code generation; strict production build passes.	Supports reproducibility and technical reliability.
iPhone 13 acceptance	Production HTTPS camera access, target tracking, direct 3D touch, all three missions, BM/English audio and Java evidence verified.	Supports readiness for live demonstration on the stated device.
AR learning study	Not yet completed.	A larger classroom pilot with task completion, error rate and pre/post measures is required before claiming measured AR learning gains.

8. Innovation Value Proposition

Dimension	Competition case
Novelty	A tracked 3D Swing prototype functions as the learning model, interaction surface and deterministic assessment interface.
Usefulness	Learners move from rule recognition to functional testing and structural repair in one short sequence.
NOSS relevance	Each mission produces observable evidence for prototype implementation and UI/UX functionality testing.
Integrity	The engine judges; AR state and generated Java are deterministic. No generative AI decides the grade.
Deployability	Runs from a secure web link on an iPhone 13 without installing a native application.
Replicability	The same pattern can be extended to additional layout managers and institution-specific prototype tasks.

9. Expected Impact

For learners, LayoutLab AR shortens the distance between an abstract constraint and its visible consequence. For lecturers, it provides a repeatable demonstration and a clear evidence trail: mission progress, repaired structure and generated Java. For TVET delivery, the web-based approach reduces installation burden while still using spatial interaction to make a difficult programming concept tangible. The immediate use case is the Java GUI curriculum at Politeknik Mukah; the design can later support broader Malaysian polytechnic and vocational programmes.

10. Competition Readiness and Next Validation

Current readiness

The competition demonstration is technically ready on the available iPhone 13: original target, tracked 3D model, three assessed missions, six media/learning elements, deterministic scoring and final Java evidence are operational at the production URL.

Before judging, the team should complete a small, documented AR pilot rather than overstate the earlier browser surveys. Recommended evidence includes: target-acquisition time, mission completion time, wrong-region attempts, successful repair rate, System Usability Scale or a short usability instrument, and one concise pre/post BorderLayout task. Screen recordings should capture the complete target, direct touches on the 3D regions, the resize animation, X-ray collision, nested JPanel repair and expanded Java evidence.

11. Conclusion

LayoutLab AR addresses a precise competency gap: learners can write Java Swing syntax yet remain unable to predict how a layout manager will construct and resize an interface. The innovation turns that invisible mechanism into a tracked, touchable and assessable prototype. Its strongest feature is not the novelty of placing a model over a camera feed; it is the integrity of the learning sequence. The learner constructs a state, tests a rule, diagnoses a failure, repairs the structure and receives deterministic Java evidence. This positions AR as the essential method of learning and assessment while keeping claims verifiable and aligned to IT-010-3:2016-C01.

Declaration

The authors declare that the work is original and that all technical and evaluation evidence presented can be verified. Any AI-assisted drafting or development output has been reviewed by the authors. Generative AI does not determine learner correctness, scoring or the assessed Java output in LayoutLab AR.

References

- Brown, N. C. C., & Wilson, G. (2018). Ten quick tips for teaching programming. *PLOS Computational Biology*, 14(4), e1006023. <https://doi.org/10.1371/journal.pcbi.1006023>
- Ishizue, R., Sakamoto, K., Washizaki, H., & Fukazawa, Y. (2020). PVC.js: Visualizing C programs on web browsers for novices. *Heliyon*, 6(4), e03806. <https://doi.org/10.1016/j.heliyon.2020.e03806>
- Keuning, H., Jeuring, J., & Heeren, B. (2018). A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1), Article 3. <https://doi.org/10.1145/3231711>
- Politeknik Mukah. (2026a). Evaluating the impact of Magik Layout on Java Swing layouts (DFP50463): Iteration 1 student survey [Unpublished internal evaluation].
- Politeknik Mukah. (2026b). LayoutLab student evaluation: Iteration 2 [Unpublished internal evaluation].
- Politeknik Mukah. (2026c). LayoutLab AR competition build, device-acceptance records and technical documentation [Unpublished technical documentation].
- Qian, Y., & Lehman, J. (2017). Students' misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education*, 18(1), Article 1. <https://doi.org/10.1145/3077618>
- Salinas, M., Leger, P., Fukuda, H., Cardozo, N., Duarte, V., & Figueroa, I. (2023). Evaluations of integrated programming environment for first-year students in computer engineering. *Journal of Universal Computer Science*, 29(1), 73–97. <https://doi.org/10.3897/jucs.81329>

Demonstration Access

AR module: <https://magik-layout.mhdazlan.cc/#/ar-lab>

Tracking target: <https://magik-layout.mhdazlan.cc/ar/layoutlab-target.png>